JavaScript & TypeScript Master Reference

Aesthetic Executive Guide & Complete Technical Architecture Manual

1. Diagnostics, Comments & Core Syntax

`console.log()` formats output directly to standard streams. Use JSDoc comments for intelligent IDE intellisense:

```
/**
 * Connects user to enterprise database socket
 * @param userId - Unique record identifier
 * @returns boolean connection status
 */
function connectUser(userId: string): boolean { return true; }
```

2. Variables, Scope & Memory Allocation

Variables reside in the **Call Stack** (primitives, pointers) or **Heap Memory** (complex objects, closures managed by GC).

Feature	var	let	const
Scope	Function Scope	Block Scope	Block Scope
Hoisting	Initialized to undefined	Temporal Dead Zone (TDZ)	Temporal Dead Zone (TDZ)
Mutability	Re-declaration allowed	Re-assignment allowed	Immutable binding reference

3. Data Types & Static Typing

TypeScript adds compile-time type verification over JavaScript's dynamic typing system without introducing runtime performance overhead.

```
type UserStatus = "ACTIVE" | "SUSPENDED" | "PENDING";
interface UserProfile {
    readonly id: string;
    name: string;
    roles: string[];
    metadata?: Record<string, unknown>;
}
```

4. Operators & Control Flow

Strict equality (`===`) prevents implicit type coercion bugs compared to loose equality (`==`). Nullish coalescing (`??`) handles fallback assignments safely without evaluating falsy values like `0` or `""` incorrectly.

```
// Nullish coalescing vs Logical OR
const count = 0;
const val1 = count || 10; // 10 (0 is treated as falsy)
const val2 = count ?? 10; // 0 (0 is preserved safely)
```

Control flow constructs include standard loops, `for...in` for enumerable keys, and `for...of` for iterable values.

5. Advanced Functions & Callbacks

Functions maintain an execution context stack. Arrow functions maintain lexical `this` bindings, making them ideal for callbacks and event handlers.

```
// Overloaded Function Contract
function formatInput(val: string): string;
function formatInput(val: number, precision: number): string;
function formatInput(val: string | number, precision?: number): string {
    return typeof val === "string" ? val.toUpperCase() : val.toFixed(precision ?? 2);
}
```

6. Arrays & High-Order Methods

Array methods enable clean, declarative data transformations:

```
const data = [10, 20, 30, 40, 50];
const doubled = data.map(x => x * 2); // [20, 40, 60, 80, 100]
const total = data.reduce((acc, curr) => acc + curr, 0); // 150
```

7. Classes, Inheritance & Interfaces

Classes compile down to standard JavaScript constructor functions while providing robust access modifiers (`public`, `protected`, `private`) and static members.

```
class EnterpriseRepository implements DatabaseClient {
    public readonly connectionURI: string;
    constructor(uri: string) { this.connectionURI = uri; }
    public async connect(): Promise<boolean> { return true; }
}
```

8. Modules & Resolution

ES Modules (ESM) provide static dependency analysis, allowing bundlers to perform efficient tree-shaking. Use `import type` to enforce complete compile-time type erasure.

Interfaces

Zero runtime footprint. Perfect for API contracts and structural typing.

Classes

Maintains runtime presence, constructor logic, and stateful methods.